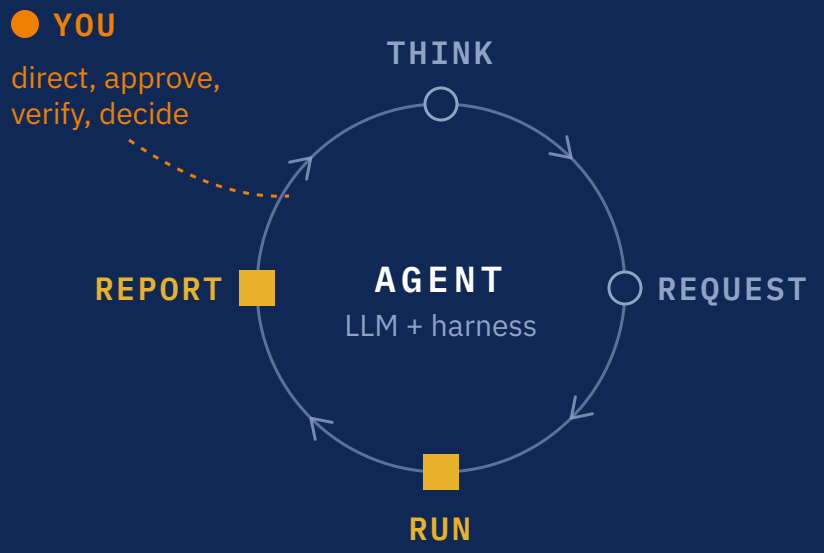
EXERCISE · WEEK 01 · TA SESSION 1

~ $ cd ~/TA1

Your workspace

Terminal · Git · Your development environment

1 terminal — 2 git — 3 the agent — 4 project



00560413 – AI Fluency and Agentic Methods for Engineers · Winter 2026–27

Ulman 501 / 703 · 10:30–12:20

OPENING · INTRODUCTION

Meet your TAs



Ori Goldfryd

A little about me

- M.Sc. student in Data Science at the Technion, with degrees in Computer Science, Law, and Economics
- Recently returned from a research stay at Cornell Tech in NYC
- My research interests include LLMs, multi-agent communication, human-AI interaction, and AI safety



Ramzy Ayan

A little about me

- I work at Amazon, Annapurna Labs, on agentic systems for the Nitro cards team
- I completed a B.Sc. in Computer Science
- I studied on exchange at GTIIT, the Technion's campus in Shantou, China
- I am an M.Sc. student in the Energy program, supervised by Assoc. Prof. Alon Grinberg Dana
- I founded Degreez, a free academic network

OPENING · YOU SET THIS UP AT HOME

What should already be on your laptop?

ITEM	WHY
Ubuntu (Windows) or Terminal (Mac)	Where you type commands. Later, the agent works here too.
Git	Saves versions of your work, so you can go back.
uv	Sets up Python for each project, so projects do not mix.
A GitHub account	Where your project lives online. Your TAs review it there.
VS Code (recommended)	An easy editor for text and code.

> **Still need help with setup?** Please read the **detailed setup instructions** carefully and follow them step by step. If you get stuck, we're happy to help during the break or in office hours. You can also try working through the setup with a classmate.

YOUR TURN **OPENING · 3 MIN**

Check your stack

```bash
git --version   # 1  2.28 or newer
uv --version    # 2  prints a version number
pwd             # 3  /home/… (Ubuntu) or /Users/… (Mac), not /mnt/c/…
# 4  Windows only, in PowerShell (not Ubuntu):
wsl -l -v       #    the VERSION column reads 2
```

Type only the command. Everything from `#` on is a note for you; on a Mac, Terminal does not ignore it.

DONE WHEN

Every line as in its comment (line 4: Windows only). uv, pwd or WSL wrong: the Pre-work page has the fix. Git older than 2.28: tell a TA in the break.

Building the course environment (`uv sync`) comes in Week 2.

OPENING · WHY THESE TOOLS

Open source: the code is public

Closed source

You get the **program**, not its code. You usually cannot read it, change it, or pass it on.

Windows, macOS, Microsoft Word

Open source

The **source code** is published under a licence that lets anyone **read, use, change and share** it.

Linux, Git, Python, uv

> Open source is not "no rules": the **licence** (MIT, Apache, GPL, ...) sets the conditions.
> And it is not only "free of charge": it is the freedom to read, change and share the code.

OPENING · WHY THESE TOOLS

Why most of our tools are open source

TOOL	OPEN SOURCE?	WHY IT MATTERS HERE
Linux (Ubuntu, in WSL on Windows)	Yes. The kernel: Torvalds, 1991. Ubuntu is one distribution of it	The same kind of terminal in WSL, on a Mac (a close Unix cousin) and on servers. The agent works in it too.
Git	Yes. Torvalds, 2005, for the Linux kernel	You can see, compare and undo every change, including an agent's.
Python and **uv**	Yes	Free; uv's lock file rebuilds the same environment anywhere.
GitHub	No: a company's service	Git is the tool; GitHub is one place to keep its repositories.

Not everything is: GitHub, Microsoft's build of VS Code and most of the language models we call are not. The course's rule: open-source and free-tier tooling wherever possible.

You can **read what a tool does**. That is the habit this course asks of you with agents: do not trust it, check it.



Much of the world's software rests on open-source projects kept up by small teams, often volunteers. One more reason to read what you install.

Headquarters



APPLE



AMAZON



LINUX

OPENING · THE WHOLE SEMESTER

How every TA session works



~10 min

Opening

A short standup on challenges submitted in advance to the TA; recent AI news; or another topic.

~80 min

Demo & your project

You apply this week's lecture to your own project.

~10 min

Wrap up

Review, commit your work, and discuss next steps.

These TA sessions are for **hands-on** practice: applying what you learn in lectures and making progress on your project each week.

From Week 2: standup 10 · your project 100 · commit and disclose 10

OPENING · TODAY'S TA SESSION

Four parts, one goal

Build the foundations you need to work with AI agents.

01
Terminal
Move around and read files

02
Git
Save versions of your work

03
The agent
How an AI agent works

04
Project
Choose your project topic

Today's session



OPENING · HOW TO READ THESE SLIDES

Two kinds of slides

WATCH

I explain a concept or demonstrate it in practice. Your task is to follow along, ask questions, and make sure you understand.

YOUR TURN

You work on your own laptop. Follow the steps on the slide and use any additional materials we provide. The slide tells you when you are done.

Stuck? Do this, in order

1. Read the error message carefully and try to solve it by yourself.
2. Quietly ask your neighbour for help.
3. Raise your hand.

PART 1 OF 4

The terminal

01

24 MIN

BY THE END OF THIS PART, YOU CAN:

- Move between folders, read files, and download a file.
- Explain why agents need commands.

WATCH PART 1 · THE TERMINAL

The terminal

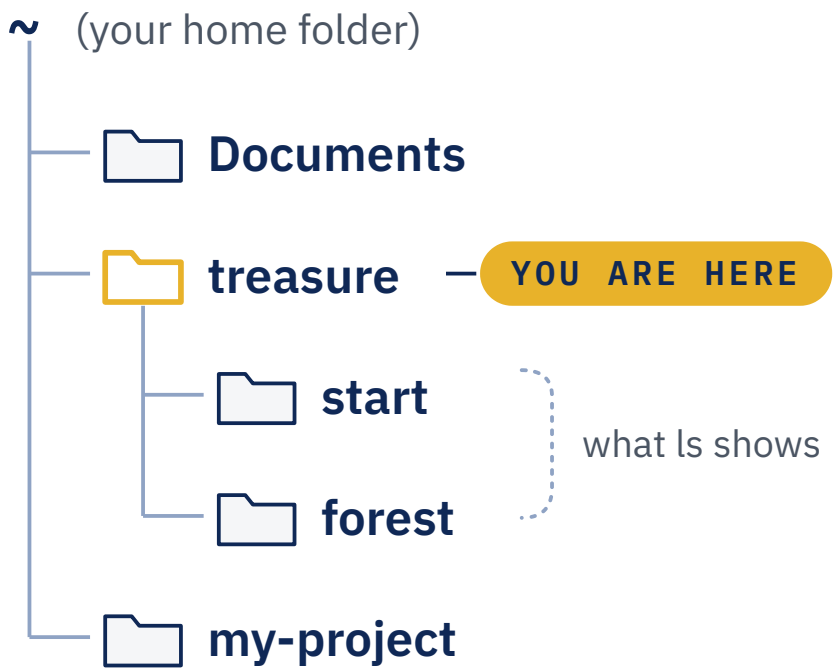
A terminal is a window where you type commands to interact with your computer.

Each command runs from a **current directory**: the folder you are in now.

> **Why learn commands?** AI agents can run them too. Understanding commands helps you check what an agent does.

WATCH PART 1 · THE TERMINAL · WHERE AM I?

Your home folder, as a tree



ILLUSTRATIVE

```
~ $ cd treasure

~/treasure $ ls

# what ls shows: see the tree

~/treasure $
```

pwd where am I?

ls what is here?

WATCH PART 1 · THE TERMINAL

Reading a command

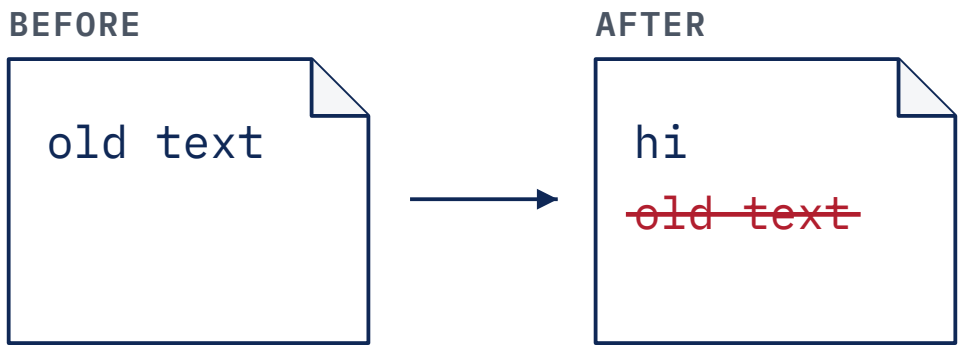
COMMAND	MEANING
`cd ~`	`~` means your home folder.
`cd ..`	`..` means the folder one level above.
`cat <file>`	Replace `<file>` with a real name, such as `README.txt`.
`echo "hi" > note.txt`	`>` writes output to a file and replaces its contents.
`echo "more" >> note.txt`	`>>` adds output at the end of the file.

WATCH PART 1 · THE TERMINAL

One arrow replaces, two arrows add

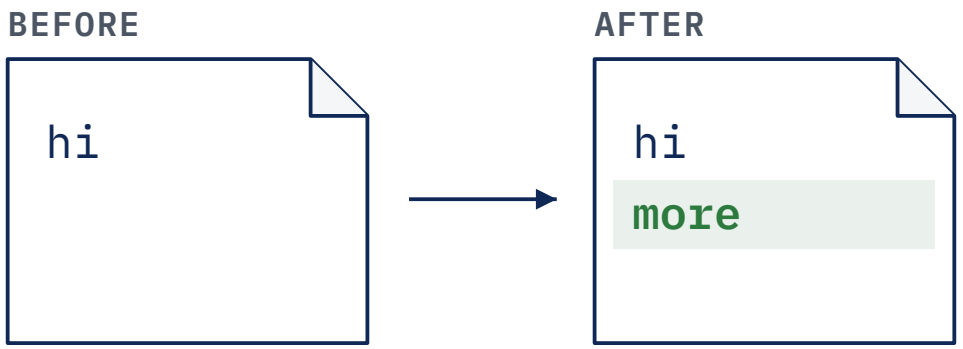
> replaces the contents

```
$ echo "hi" > note.txt
```



>> adds at the end

```
$ echo "more" >> note.txt
```



WATCH PART 1 · THE TERMINAL

Twelve commands, three purposes

Look around

Where am I, what is here?

- `pwd` — Where am I?
- `ls` — What is in this folder?
- `cd <folder>` — Go into a folder
- `cd ..` — Go up one folder

Change things

Make, copy, move, delete.

- `mkdir <name>` — Make a folder
- `cp a b` — Copy a file
- `mv a b` — Move or rename a file
- `rm file` — Delete without a recycle bin

Read and get

Read, write, download, run.

- `cat <file>` — Show what is inside
- `echo "hi" > file` — Write text into a file
- `curl -fLO URL` — Download a file
- `bash script.sh` — Run a script

Why these commands? They cover today's tasks. You'll learn more as you need them.

WATCH PART 1 · THE TERMINAL · WHAT WILL HAPPEN? SAY IT, THEN CLICK

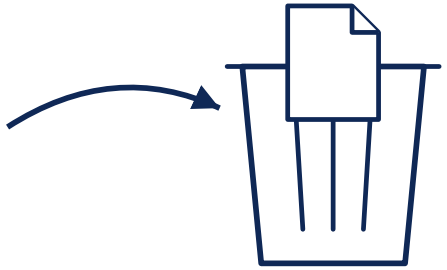
Predict, then run

```
pwd                                  # where am I?
ls                                   # what is here?
mkdir demo                           # make a folder
cd demo                              # go inside
echo "hello" > hello.txt             # write a file
cat hello.txt                        # read it
cd ..                                # go up
rm demo/hello.txt                    # delete: no recycle bin!
ls demo                              # empty now
```

W01 · PART 1

Expectation

```
$ rm demo/hello.txt
```

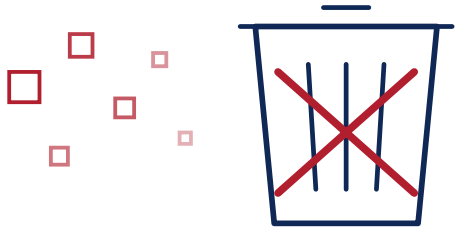


Safe in the bin.

I can get it back.

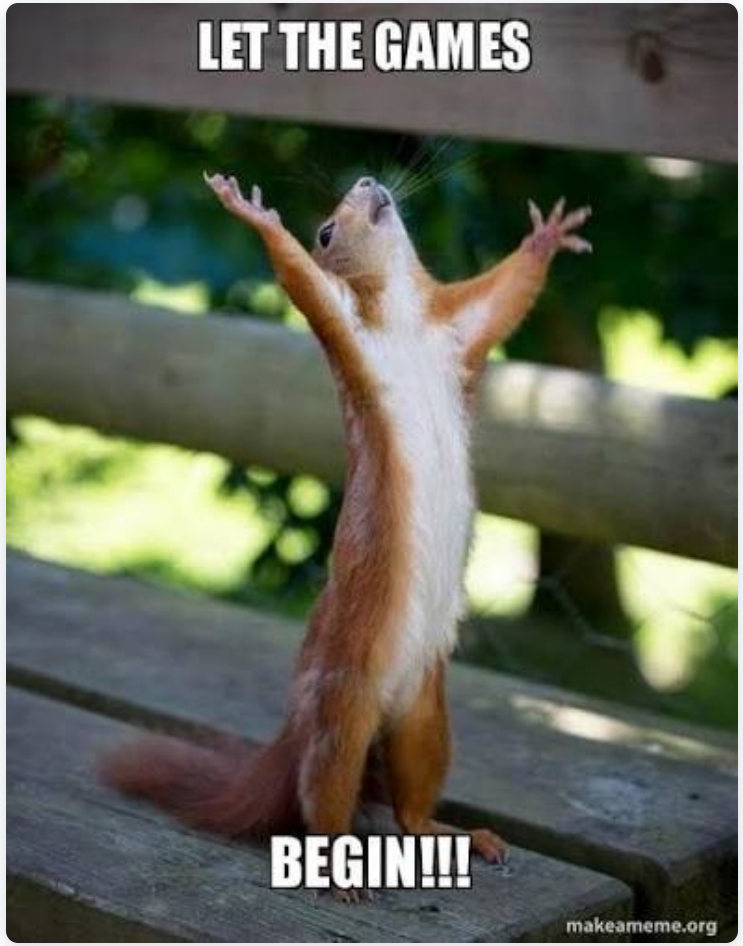
Reality

```
$ rm demo/hello.txt
```



Gone.

No recycle bin.



Laptops open.

YOUR TURN **PART 1 · THE TERMINAL · 14 MIN**

The treasure hunt

BASH

```bash
mkdir -p ~/TA1                  # 1  Create and enter the TA1 folder
cd ~/TA1
# 2  Download the setup script
curl -fLO \
https://technionaifluency.github.io/www/exercises/w01/setup_treasure.sh
bash ~/TA1/setup_treasure.sh    # 3  Build the treasure folder
cd ~/TA1/treasure               # 4  Go in and read the message
cat README.txt
# 5  Follow the clues. Use ls, cd, cat, mkdir, mv.
```

Type the command, not the # note.

DONE WHEN

The key is in `done` and no longer in `mountain`.

BACKUP PART 1 · THE TERMINAL · ONLY IF THE DOWNLOAD FAILS

If the download fails

BASH

```bash
# 1  Save setup_treasure.sh from the course website (Exercises, W01) to Downloads
# 2  Windows: copy it to Ubuntu. NAME is your Windows user (see: ls /mnt/c/Users/)
cp "/mnt/c/Users/NAME/Downloads/setup_treasure.sh" ~/
# 3  Mac: copy it to your home folder
cp ~/Downloads/setup_treasure.sh ~/
# 4  Check that it is there
ls ~
# 5  Then go on with the game. Skip the curl step.
bash ~/setup_treasure.sh
```

DONE WHEN

`ls ~` shows `setup_treasure.sh`. Then go back to the game slide.

PART 2 OF 4

Git: save points

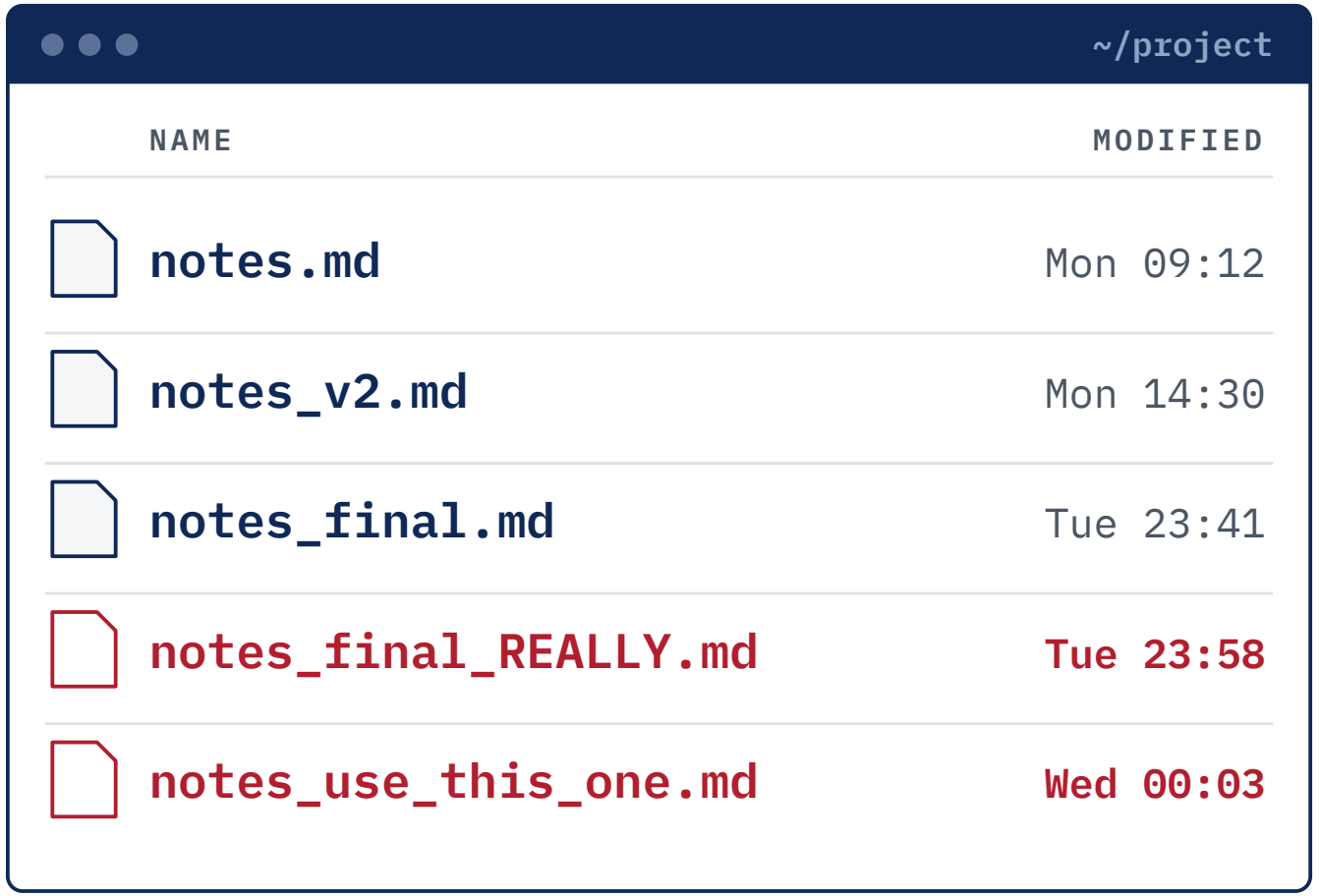
02

29 MIN

BY THE END OF THIS PART, YOU CAN:

- Explain what a save point is, and why we use them.
- Save versions, switch between existing branches, and undo an edit.

W01 · PART 2



Which one is right?

What changed?

Five files. No labels.

WATCH PART 2 · GIT

Why Git?

Without Git

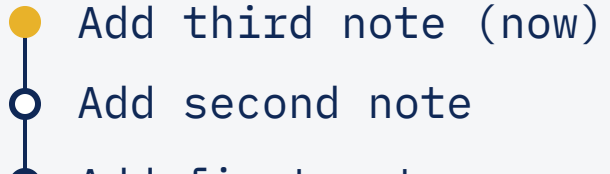
TEXT

```
notes.md
notes_v2.md
notes_final.md
notes_final_REALLY.md
notes_use_this_one.md
```

Which one is right? What changed?

With Git

notes.md

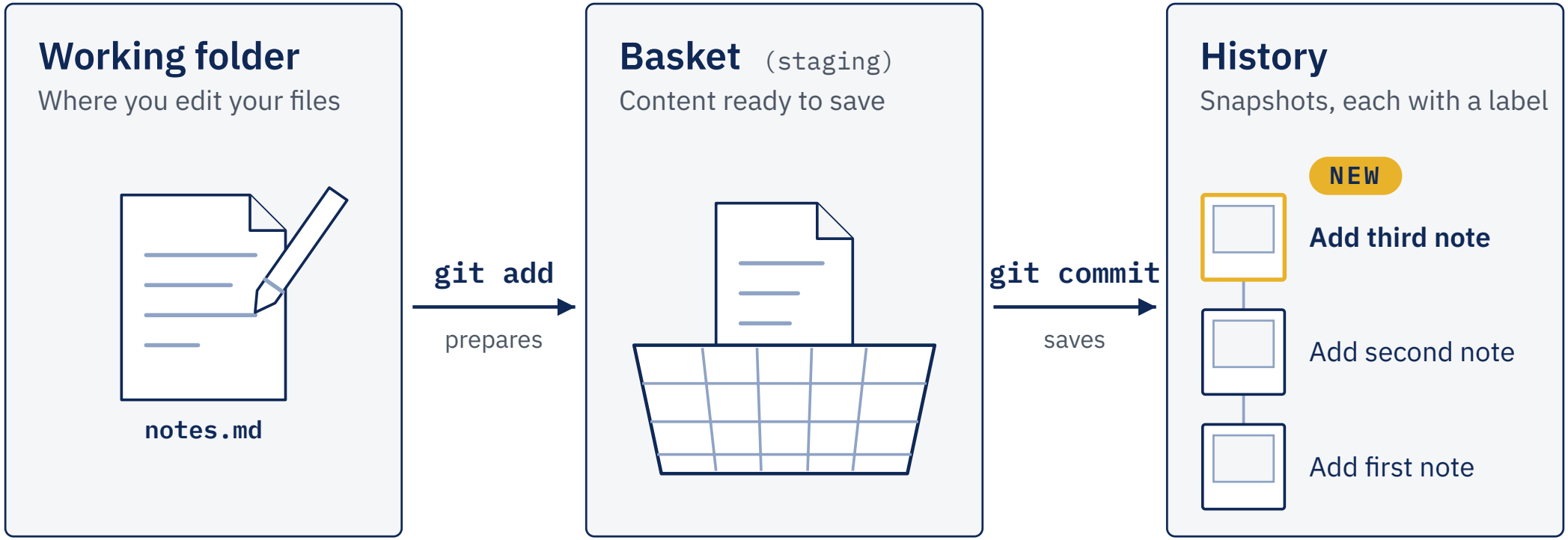


One file. Every version is saved, with a label. You can go back.

Git saves versions of your project when you make a **commit**. You can compare changes and return to an earlier saved version.

WATCH PART 2 · GIT

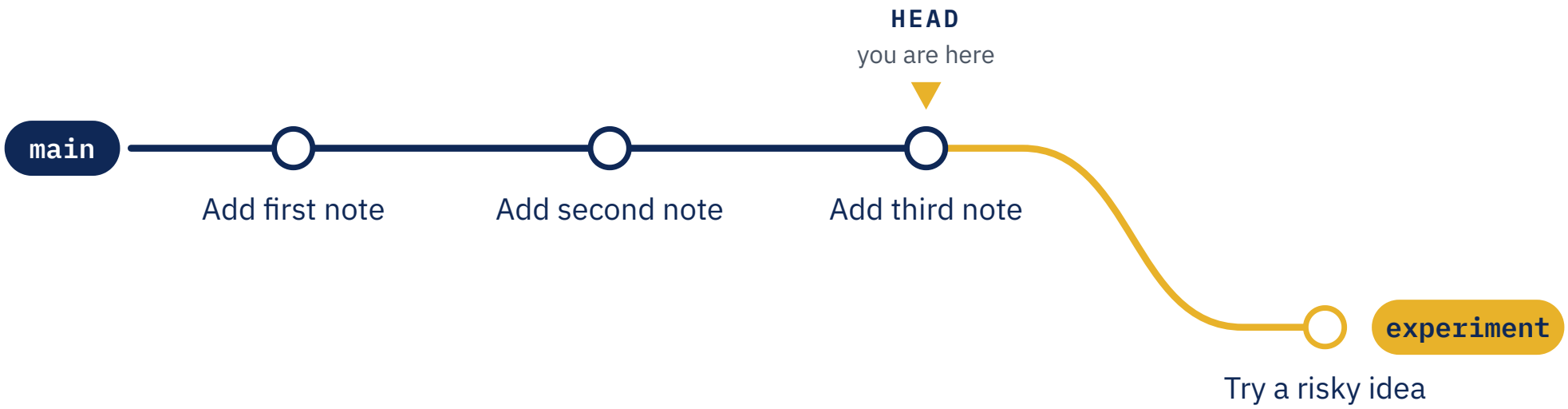
How Git saves a version



`git add` prepares the changes. `git commit` saves the version.

WATCH PART 2 · GIT

Branches: separate lines of commits



A branch is not a sandbox. It keeps commit history separate; it does not isolate running programs, untracked files, or uncommitted changes.

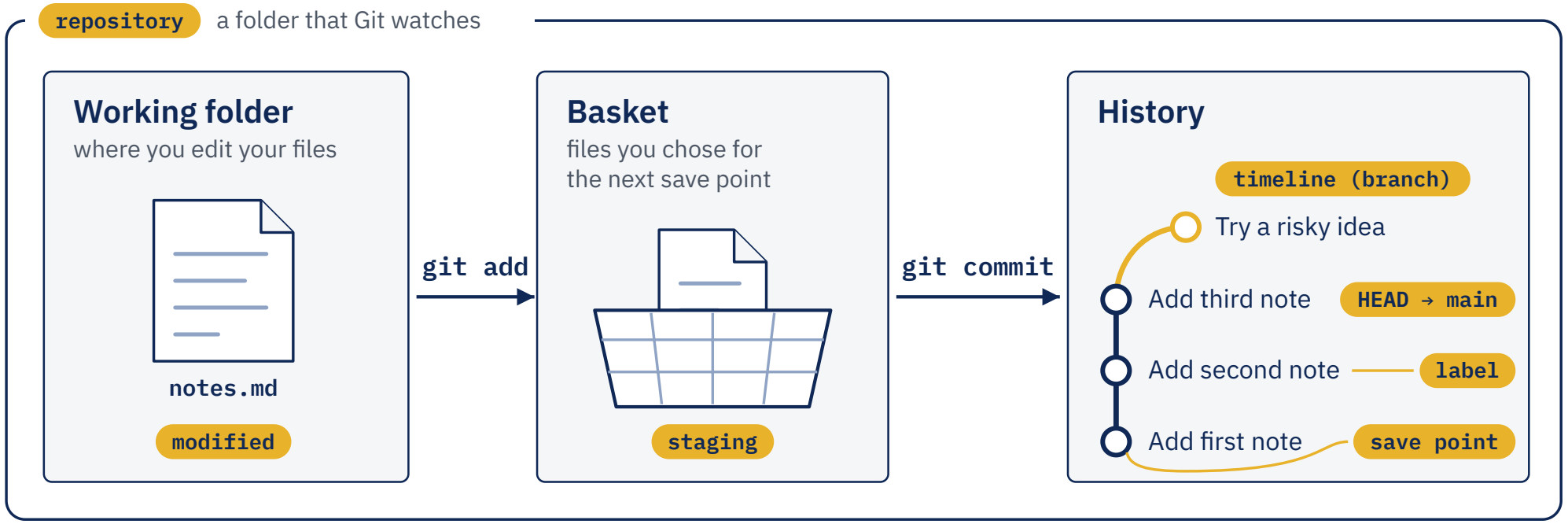
WATCH PART 2 · GIT

The words of Git

TERM	MEANING
Repository	A folder that Git watches
Save point (commit)	A photo of your files, with a label
Label	A short sentence that says what changed
Timeline (branch)	A separate line of save points (the first is `main`)
Basket (staging)	The files you chose for the next save point
Untracked, modified	New file not saved yet, saved file that changed
HEAD	A marker: you are here

WATCH PART 2 · GIT

Git on one page



How a save point is made:

1. Edit a file.
2. **`git add`**: put it in the basket.
3. **`git commit`**: take the photo, with a label.

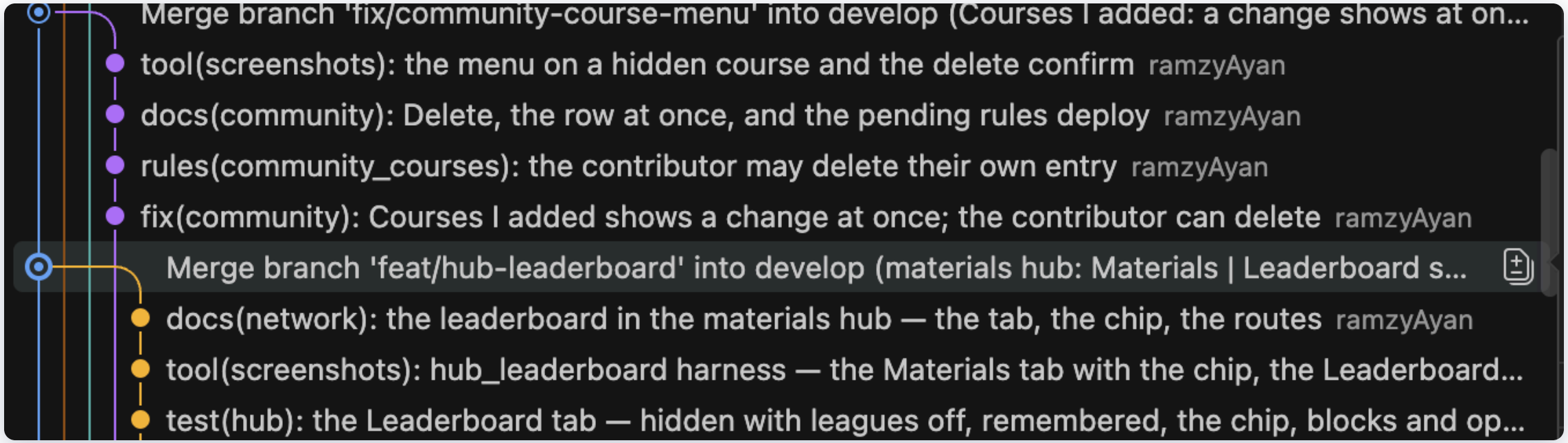
WATCH PART 2 · GIT

Git commands, in plain words

COMMAND	PLAIN WORDS
`git init`	Create a repository here
`git status`	What does Git see right now?
`git add <file>`	Put the file in the basket
`git commit -m "label"`	Take the photo, with a label
`git log --oneline`	Show commits on this branch
`git branch`	Show all timelines
`git switch <name>`	Travel to another timeline
`git diff`	Show exactly what changed
`git restore <file>`	Discard unstaged edits to this file

WATCH PART 2 · GIT · IN A REAL PROJECT

What branches look like in a real project



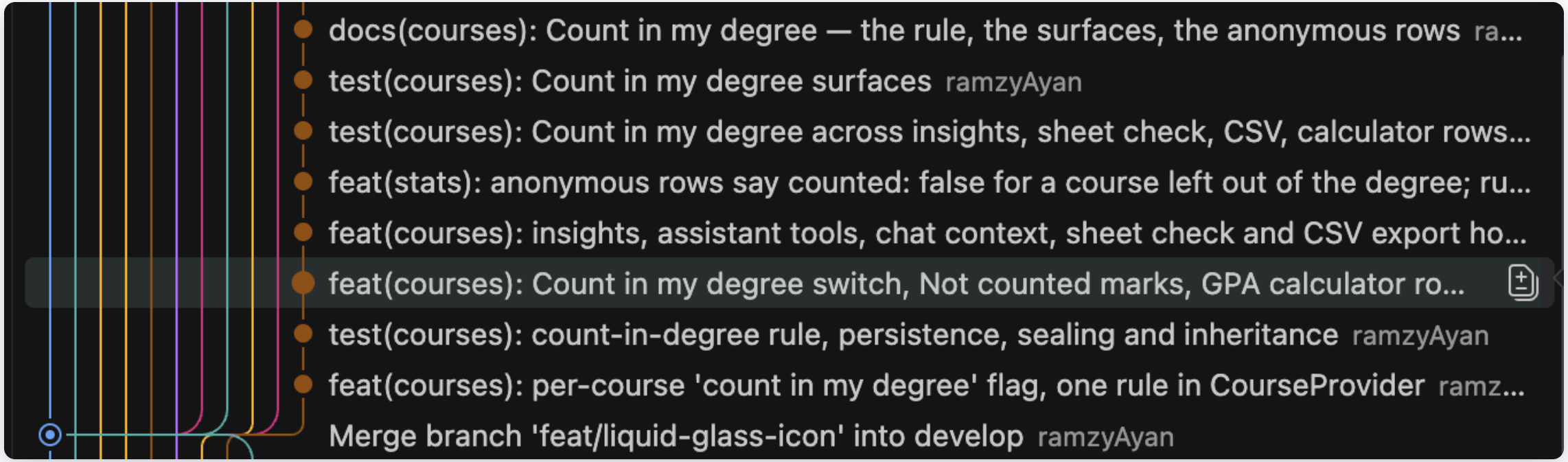
Each dot is a save point (commit), with a **label** that says what changed: `test(hub): …`

Each coloured line is a timeline (branch): several pieces of work, side by side, none touching the others

"Merge branch … into develop": a finished timeline joins back. Merging is Week 2.

WATCH PART 2 · GIT · IN A REAL PROJECT

The same project, on a busier day



Each coloured line is work in progress, kept apart until it is merged. The brown one is one feature in **eight labelled save points**.

YOUR TURN PART 2 · GIT · 19 MIN

Git Quest: four levels

1

Look around

`git status`

`git log --oneline`

Find the save point whose label starts with KEY.

2

Travel between timelines

`git branch`

`git switch`

Follow the KEY to another timeline. One timeline is a wrong turn.

3

Make your own save point

`git add`

`git commit`

Write your name in explorer.txt, then commit it.

4

Undo a mistake

`git diff`

`git restore`

Undo the vandal's line in treasure-map.txt.

Start with `cat START-HERE.txt`. Open each level file only when it tells you to.

YOUR TURN **PART 2 · GIT · 19 MIN**

Git Quest: start the game

```bash
cd ~/TA1                          # 1  Go to your TA1 folder
# 2  Download the setup script
curl -fLO \
https://technionaifluency.github.io/www/exercises/w01/setup_git_quest.sh
bash ~/TA1/setup_git_quest.sh     # 3  Build the quest folder
cd ~/TA1/git-quest                # 4  Go inside the game folder
cat START-HERE.txt                # 5  Read the first message
                                  #    Then follow all 4 levels.
```

Type the command, not the # note.

DONE WHEN

Run `bash ~/TA1/git-quest-check.sh`. It prints `FINAL STATE CHECK PASSED`. Explain to yourself a few of the self-check terms it lists.

BACKUP PART 2 · GIT · ONLY IF THE DOWNLOAD FAILS

If the download fails

BASH

```bash
# 1  Save setup_git_quest.sh from the course website (Exercises, W01) to Downloads
# 2  Windows: copy it to Ubuntu. NAME is your Windows user (see: ls /mnt/c/Users/)
cp "/mnt/c/Users/NAME/Downloads/setup_git_quest.sh" ~/
# 3  Mac: copy it to your home folder
cp ~/Downloads/setup_git_quest.sh ~/
# 4  Check that it is there
ls ~
# 5  Then go on with the game. Skip the curl step.
bash ~/setup_git_quest.sh
```

DONE WHEN

`ls ~` shows `setup_git_quest.sh`. Then go back to the game slide.

BREAK

10 minutes

11:28 ▭▭▭▭▭▭▭▭▭▭ 11:38

Back at 11:38.

During the break: come to the TA's computer and type your GitHub username next to your ID number.

PART 3 OF 4

How an agent works

03

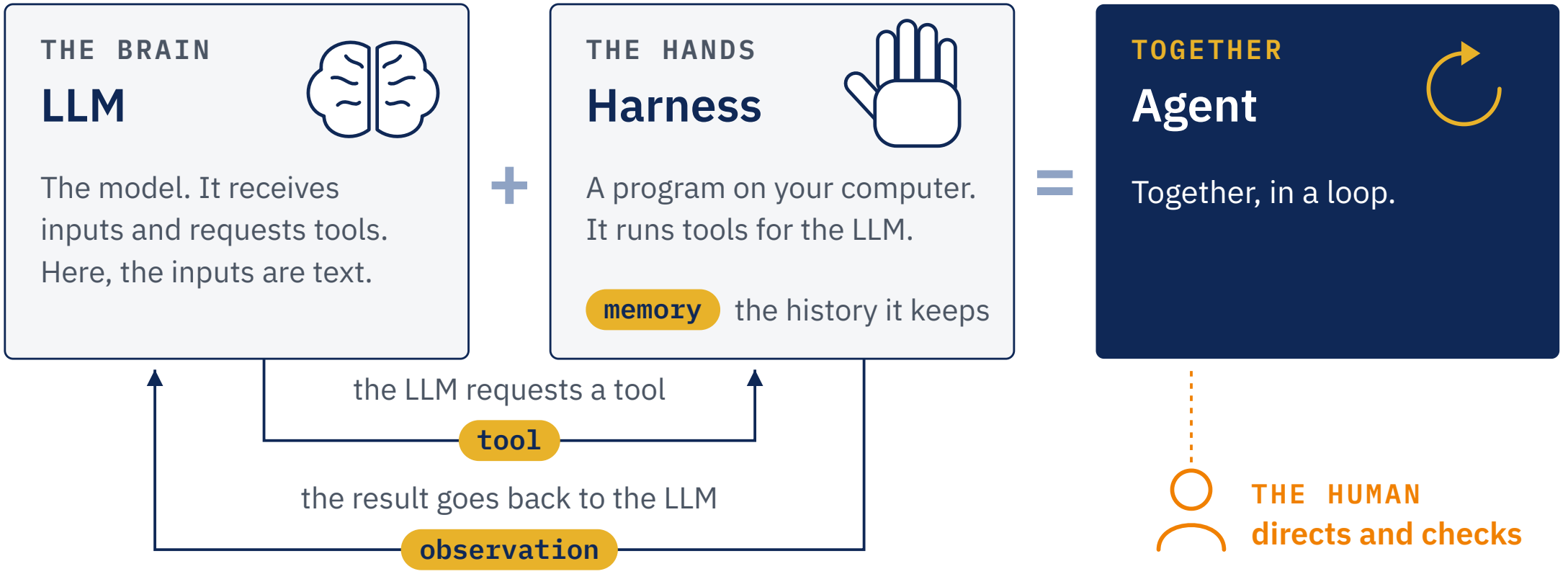
22 MIN

BY THE END OF THIS PART, YOU CAN:

- Explain what an AI agent is made of, and what the human does.

WATCH PART 3 · THE AGENT

The brain and the hands



WATCH PART 3 · THE AGENT

The brain and the hands

The LLM thinks.

The harness acts.

The human directs and checks.

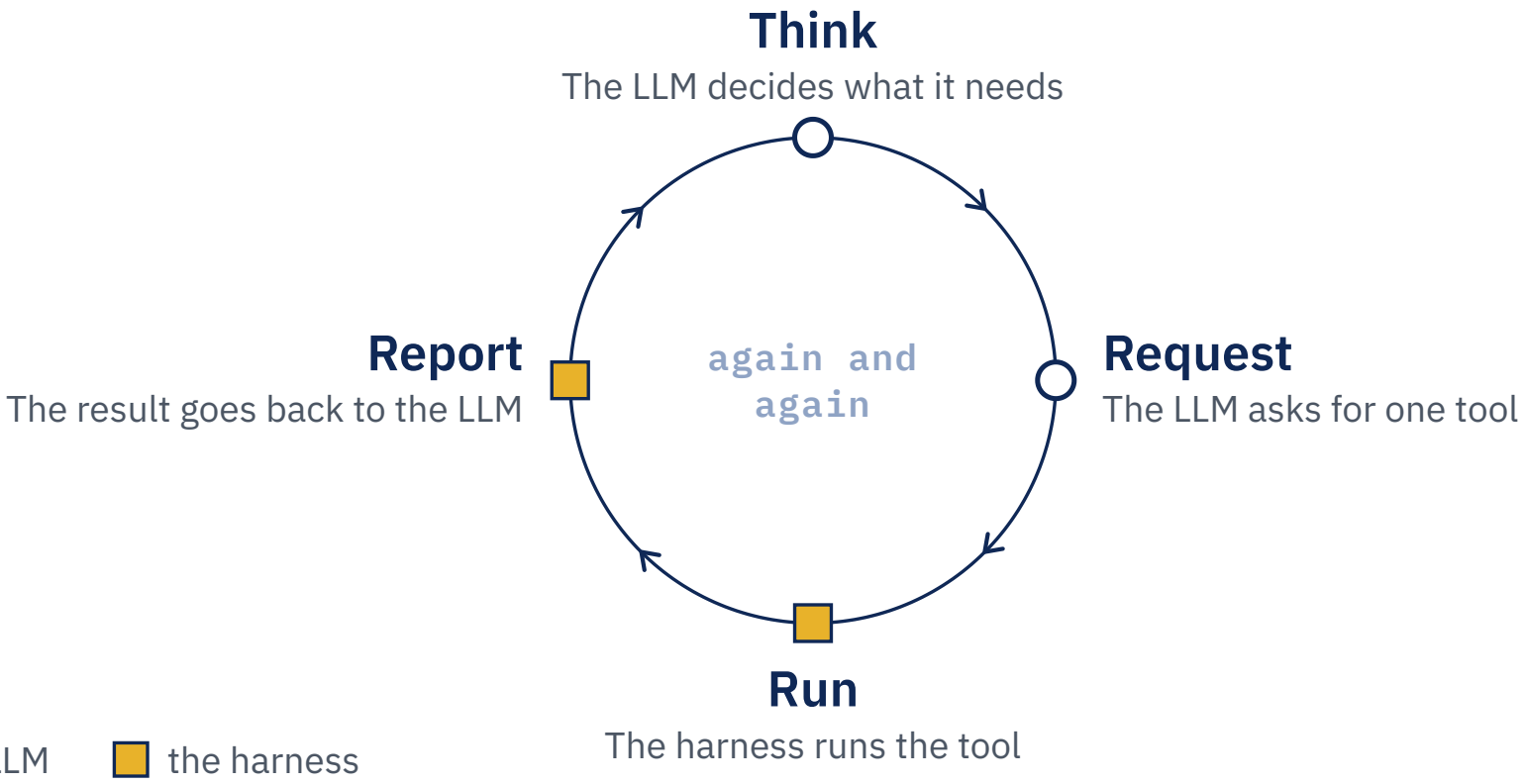
We learn more about agents in Weeks 2 & 3.

Words you will meet

- **Tool:** a command the harness can run.
- **Observation:** the result that goes back to the LLM.
- **Memory:** the history the harness keeps.

WATCH PART 3 · THE AGENT

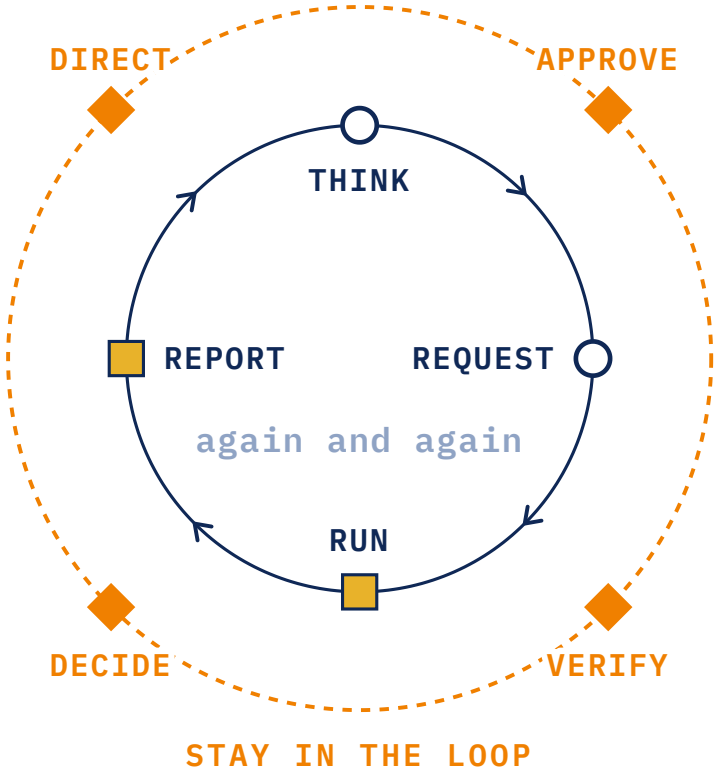
The agent loop



Agents may repeat this many times, by themselves.

WATCH PART 3 · THE AGENT

The human in the loop



YOU, THE HUMAN

- **Direct**
  Give the agent a clear task
- **Approve**
  Check requests before approving
- **Verify**
  Check the result against your goal
- **Decide**
  Keep it, or undo it (Git!)

You are responsible for what the agent does.

YOUR TURN PART 3 · THE AGENT · ANSWER FIRST, THEN CLICK

Working with an agent: check yourself

The model requests `ls`. Who runs it?

> The harness runs the command and returns the file list to the model. The human directs and checks.

What do you check before approving a file change?

> Read the command. Check which file it changes and whether the action matches your goal.

The agent says 'Done'. How do you verify?

> Read the file and compare it with the task. For a tracked file, inspect `git diff`.

How can Git help undo a mistake?

> Commit the working version first. For a tracked file, `git restore` can discard an unstaged edit.

WATCH PART 3 · THE AGENT

Your harness: next step

NOW

Today

Nothing to install. Basic understanding.

NEXT

Before Week 3

Instructions come in the Week 2 handout.
Install it at home.

PART 4 OF 4

Your project topic

04

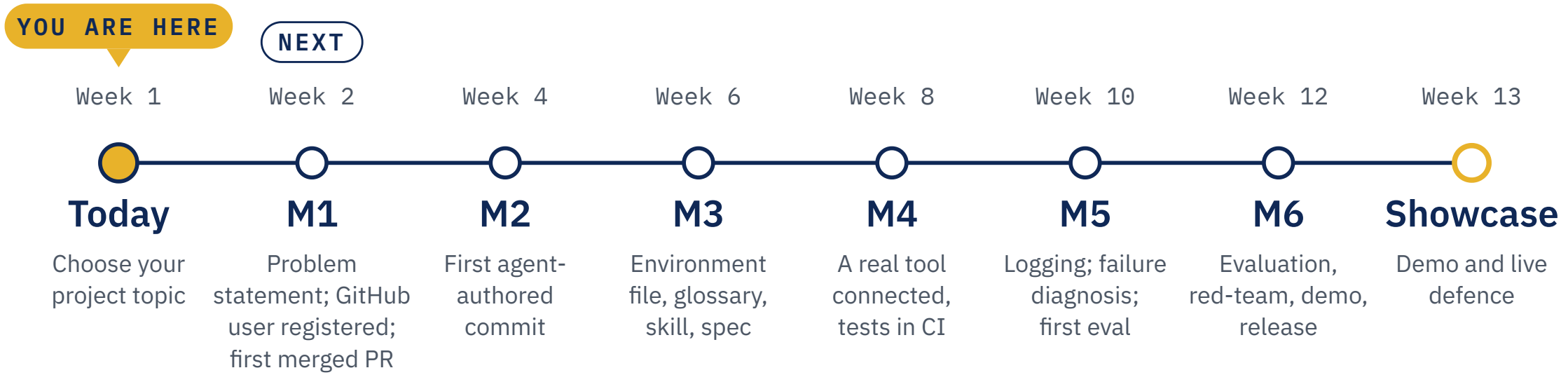
15 MIN

BY THE END OF THIS PART, YOU CAN:

- Choose a real engineering or research problem.
- Find the simulator that could compute the answer.
- Start your proposal today, and finish it at home.

WATCH PART 4 · YOUR PROJECT

Where we are in your project



Today: a topic draft. Full milestone requirements are on the course page.

WATCH PART 4 · YOUR PROJECT

Your project, in three questions

1

The problem and goal

What should improve? How will you measure it? What limits matter?

2

The simulator

A program that computes how a system behaves. You give it inputs; it returns results.

3

The agent's job

What will the agent decide or change? Why might it need reasoning?

In your proposal template: 1. The problem and goal · 2. The simulator · 3. The agent's role

WATCH PART 4 · YOUR PROJECT

A clear topic, and a vague one

Too vague

"I want to use AI to improve traffic."

Clear

"Cars wait too long at one busy junction. I want to find **traffic-light timings** that **reduce the average waiting time**."

A clear topic says **what changes**, and **which result improves**.

WATCH PART 4 · YOUR PROJECT

One full example

1
The problem

Cars wait too long at a busy junction. We want to reduce their waiting time.

2
The simulator

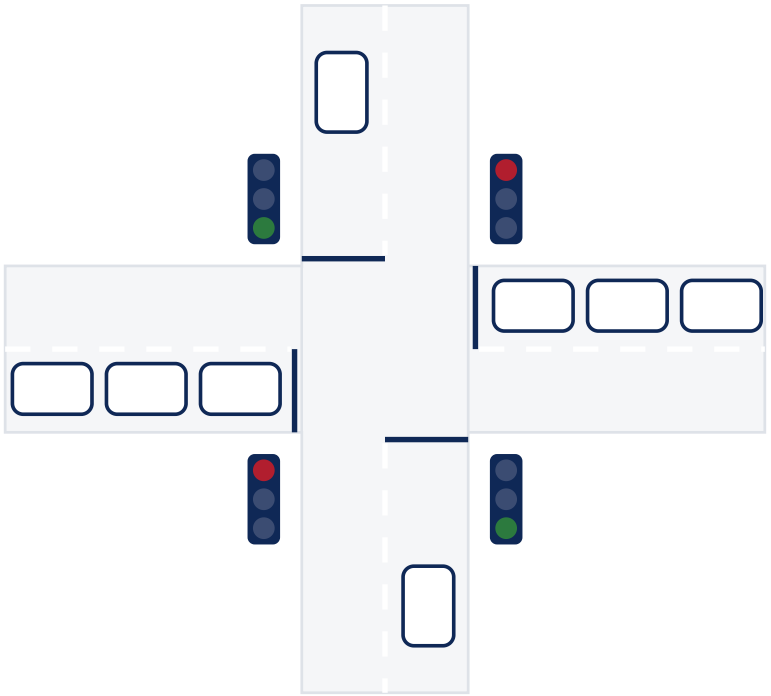
A program that simulates traffic at the junction. We set how long each light stays green. It calculates how long cars wait.

3
The agent's job

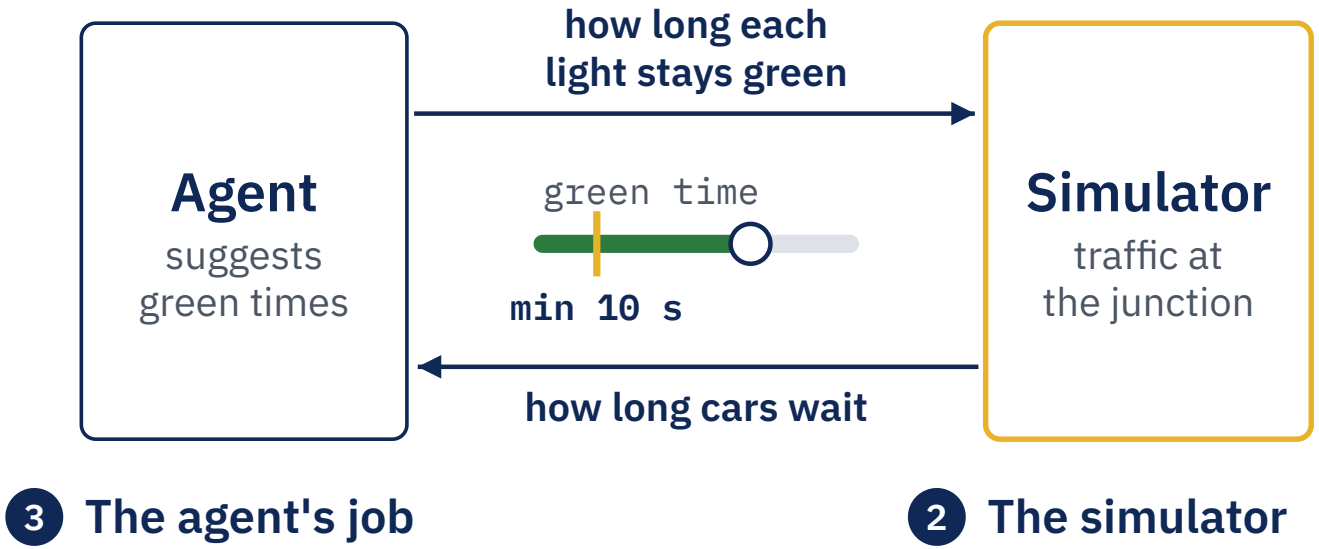
Suggest how long each light should stay green. Use the simulator to check whether waiting times improve. Each green light must last at least 10 seconds.

WATCH PART 4 · YOUR PROJECT

One full example



1 The problem



3 The agent's job

2 The simulator

WATCH PART 4 · YOUR PROJECT

Does this task need an LLM?

A fixed search may be enough

- Try a fixed set of traffic-light timings.
- Run the simulator for each option.
- Choose the option with the shortest waiting time that meets the constraints.

An LLM might also...

- Turn a written request, such as "give buses priority", into goals and constraints.
- Use the results to suggest and explain the next experiment.
- Help adapt the plan when the requirements change.

The for-loop test: Remove the LLM. Can a simple search solve the task just as well?

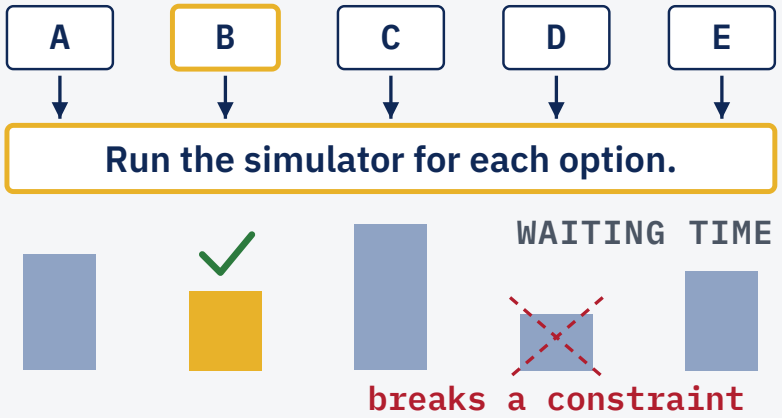
A possible role is not enough. We need to test whether the LLM actually helps.

WATCH PART 4 · YOUR PROJECT

The for-loop test

A fixed search may be enough

Try a fixed set of traffic-light timings.



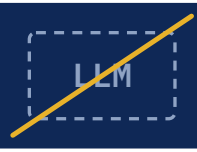
Choose the option with the shortest waiting time that meets the constraints.

An LLM might also...

1. Turn a written request, such as “give buses priority”, into goals and constraints.
2. Use the results to suggest and explain the next experiment.
3. Help adapt the plan when the requirements change.

THE FOR-LOOP TEST

Remove the LLM. Can a simple search solve the task just as well?



YOUR TURN **PART 4 · YOUR PROJECT**

Draft your project proposal

BASH

```
mkdir -p ~/my-project/docs     # 1  Make a project folder, start Git
cd ~/my-project
git init -b main
# 2  Download the proposal template
curl -fL -o docs/project-proposal.md \
https://technionaifluency.github.io/www/exercises/w01/project-proposal.md
# 3  Write each answer in place of its <!-- … --> line; save
code ~/my-project/docs/project-proposal.md
```

Type the command, not the # note.

DONE WHEN

`grep -c '<!--' docs/project-proposal.md` prints less than 4 today (each answer replaces one comment); 0 before your homework commit.

YOUR TURN **WRAP-UP · SAVE IT**

Save your proposal in Git

```bash
cd ~/my-project                          # 1  Go to your project folder
git add docs/project-proposal.md         # 2  Save your proposal in Git
git commit -m "Add project proposal"
git log --oneline                        # 3  Check your commit and status
git status
```

Type the command, not the # note. `Author identity unknown`? Inside `~/my-project`: `git config user.name "Your Name"` and `git config user.email "you@example.com"`, then commit again.

> **DONE WHEN**
>
> Your commit is in the history, and the working tree is clean.

Today: Your proposal is saved in Git on your computer.
Next week: We will learn GitHub and upload your proposal there. The course staff will review and grade it on GitHub.

WHAT WE DID

You now have

- ✓ A terminal you can use without fear
- ✓ Saved versions and practice switching branches
- ✓ The agent loop, and your four jobs in it
- ✓ Start your project topic

BEFORE THE NEXT TA SESSION

Homework

1. Finish `docs/project-proposal.md`, then commit it
2. If you did not yet: type your GitHub username on a TA's computer (missed it? a TA in the Week 2 break)
3. Check that everything from today still works on your laptop

> CATCHES PEOPLE
>
> Do not download a fresh template over your completed answers. Finish the draft already in `docs/`.

NEXT TA SESSIONS

What comes next

TOPIC	WHAT
GitHub	Fork, clone, pull and push
Merge conflicts	What they are and how to fix them
Context window	What the model can see, and how to manage it
Harnesses	A first look at OpenCode, Antigravity, Claude Code and Codex
Setup Gate	The sandbox check script, shown live by a TA